CSCI 2406: Topic 07

Branching and Iteration

Comparison instructions, condition flags, conditional/unconditional branches, and control flow translation in AArch64

Computer Organization & Architecture | Topic 7

Topic 7 Outline

Topic Objective

Analysis of program control flow manipulation in AArch64, mapping condition codes, comparison flags, branching mechanisms, and high-level loops/selections into assembly.

- **Section 1**: Program Control Flow and the Program Counter (`PC`)
- **Section 2**: Comparison Instructions and Condition Flags (`NZCV`)
- **Section 3**: Branch Instructions and Specialized Opcodes
- **Section 4**: Translating Selection Structures (If-Else)
- **Section 5**: Translating Loops and Iteration Constructs
- **Section 6**: Summary and Worked Self-Test Exercises

The Mechanics of Execution Sequencing

Sequential vs. Divergent Execution

Normally, the CPU fetches and executes instructions sequentially, advancing the Program Counter (PC) by 4 bytes per instruction. Control flow instructions break this sequence to implement decision-making and repetition.

- **Sequential Flow**: $\texttt{PC} \leftarrow \texttt{PC} + 4$ (automatic instruction pacing).
- **Divergent Flow**: Target addresses rewrite the PC directly, causing a jump (branch) to a new execution routine.
- Essential for building conditional logic, functions, and iterative loops.

Section 01

Control Flow and the Program Counter

The Role of the Program Counter (PC)

The Program Counter governs execution flow within the processor datapath:

- Holds the memory address of the instruction currently being executed or fetched.
- Fixed 32-bit instruction encoding means sequential instructions are always spaced by 4 bytes.
- **Branch Target**: When a branch is taken, the target address (typically represented via symbolic labels) is loaded directly into the PC.

Branch Taken vs. Not Taken

If a branch condition holds, the branch is **taken** (PC updates to the target). If the condition fails, the branch is **not taken** and execution proceeds sequentially.

Symbolic Labels as Branch Targets

Hardcoding absolute memory addresses in assembly is error-prone and inflexible:

- **Symbolic Labels** (e.g., `loop:`, `exit:`, `else_block:`) provide human-readable names for target destinations.
- **The Assembler's Role**: Replaces labels with numerical offsets relative to the current Program Counter (`PC`).
- **Position Independence**: Allows code blocks to be relocated anywhere in memory without breaking branch references.

Two-Pass Assembly Resolution

1. **Pass 1**: The assembler scans the source, records every label's relative distance, and builds a symbol table.
2. **Pass 2**: It computes the exact instruction-byte offsets and encodes them into the branch instruction word.

PC-Relative Branch Offset Limitations

Branch instructions calculate targets relative to the current Program Counter:

- Unconditional branches (`b`) utilize a wide 27-bit signed offset, supporting jumps of ± 128 MiB within the text segment.
- Conditional branches (`b.cond`) use a compact 19-bit offset (± 1 MiB).

Sample Calculation (Conditional Branch)

A 19-bit signed offset covers $2^{19} = 524,288$ instructions ($\pm 262,144$ instructions from the `PC`). Since each instruction is 4 bytes:

$$\pm 262,144 \times 4 \text{ bytes} = \pm 1,048,576 \text{ bytes} = \pm 1 \text{ MiB}.$$

Section 02

Comparison Instructions and Condition Flags

Condition Flags: The NZCV Register

AArch64 maintains processor state flags within the PSTATE register, updated explicitly by status-setting instructions:

- **N (Negative)**: Set if the result is negative (Bit 63 of a 64-bit result is 1).
- **Z (Zero)**: Set if the result evaluates exactly to 0x0.
- **C (Carry)**: Set on unsigned arithmetic overflow or valid carry-out.
- **V (Overflow)**: Set on signed two's complement arithmetic overflow.

Example: Flag State After Subtraction

If calculating 5 - 5, the result is 0, so the **Z (Zero) flag** is set to 1. If calculating 3 - 5, the result is −2, setting the **N (Negative) flag** to 1.

Evaluating Conditions: The `cmp` Instruction

Decision-making begins by evaluating operands without destroying data:

Semantics of `cmp`

```
cmp x1, x2
```

Performs an internal subtraction (operand1 − operand2), discards the numeric result, and updates the `NZCV` condition flags based on the outcome.

- Equivalent to executing `subs xzr, x1, x2` using the hardwired zero register.
- Subsequent conditional instructions inspect the `NZCV` flags to determine control flow paths.

Explicit Flag Updates with Arithmetic Opcodes

Comparison is not restricted to the dedicated `cmp` instruction:

- Arithmetic instructions with an `s` suffix (such as `adds`, `subs`, or `ands`) perform the computation *and* update the `NZCV` flags simultaneously.
- `cmp x0, x1` is actually just an assembler alias for `subs xzr, x0, x1` (subtracting into the hardwired zero register).

Loop Decrement Pattern

```
subs x0, x0, #1      // Decrements counter in x0 and updates flags
b.ne loop_start      // Inspects Z flag; loops if x0 ≠ 0
```

Section 03

Branch Instructions and Specialized Opcodes

Unconditional Branch: The `b` Instruction

Syntax and Semantics

```
b label
```

Unconditionally redirects program execution to the specified target label, updating the PC via a 27-bit PC-relative offset.

- Used for implementing unconditional jumps, loop retries, and escaping control structures.
- Completely independent of condition flags.

Conditional Branching Syntax (`b.cond`)

Conditional branches inspect the `NZCV` flags set by a preceding `cmp` instruction:

- `b.eq label` $\rightarrow$ Branch if **Equal** ($Z == 1$)
- `b.ne label` $\rightarrow$ Branch if **Not Equal** ($Z == 0$)
- `b.lt label` $\rightarrow$ Branch if **Less Than** (signed)
- `b.ge label` $\rightarrow$ Branch if **Greater than or Equal** (signed)
- `b.hi label` $\rightarrow$ Branch if **Higher** (unsigned greater than)
- `b.ls label` $\rightarrow$ Branch if **Lower or Same** (unsigned)

Conditional Execution Flow Example

Translating a simple equality check:

C Code Segment

```
if (a == b) {
  x = 1;
} else {
  x = 2;
}
```

AArch64 Translation

```
cmp x1, x2
b.ne else_block
mov x0, #1
b end_if
else_block:  mov x0, #2
end_if:
```

Optimized Branching: `cbz` and `cbnz`

AArch64 provides specialized compare-and-branch instructions for checking zero status:

- **`cbz Rt, label`** (Compare and Branch on Zero): Branches to label if register `Rt` equals zero.
- **`cbnz Rt, label`** (Compare and Branch on Non-Zero): Branches to label if register `Rt` is non-zero.

Architectural Advantage

- Combines a register test and a conditional branch into a single instruction.
- **Does not affect condition flags** (`NZCV` remain untouched), saving instruction overhead in loops and pointer checks.

Bit-Level Branching: `tbz` and `tbnz`

For testing individual bit positions without full register comparisons:

- **`tbnz Rt, #imm, label`**: Test bit number `imm` in register `Rt`; branch to label if the bit is non-zero (1).
- **`tbz Rt, #imm, label`**: Branch if the specified bit is zero.

Use Case

Ideal for inspecting hardware status flags, bitfields, or permission bits packed into integers without requiring separate masking and comparison steps.

Section 04

Translating Selection Structures (If-Else)

Translating Selection Structures (If-Else)

Branching logic handles mutual exclusion in conditional statements:

- **Design Rule**: Assembly must invert the high-level condition to jump *over* the true block when the condition fails.
- Minimizes execution overhead and eliminates unnecessary jumps.

General Assembly Pattern

1. Evaluate condition using `cmp`.
2. Branch to alternate path on condition failure (`b.ne` / `b.le`).
3. Execute true block, then unconditionally branch past the false block (`b`).

Handling Nested Selection Logic

Complex logical conditions (such as `if (a > 0 && b < 5)`) require chained evaluations:

- **Short-Circuit Evaluation**: If the first condition fails, execution immediately jumps to the failure block without evaluating subsequent conditions.
- Implemented via sequential comparison and conditional branch blocks, matching standard compiler optimization strategies.

Assembly Implementation Strategy

```
cmp x1, #0      // Evaluate first condition (a > 0)
b.le fail_label // If a ≤ 0, short-circuit and exit
cmp x2, #5      // Evaluate second condition (b < 5)
b.ge fail_label // If b ≥ 5, exit
// True Block Code Here...
```

Section 05

Translating Loops and Iteration Constructs

Translating While Loops

High-level iteration constructs map cleanly to assembly structures:

C Code

```
while (i > 0) {
  i-;
}
```

AArch64 Assembly Translation

```
loop_start:
  cmp x0, #0
  b.le loop_end
  sub x0, x0, #1
  b loop_start
loop_end:
```

- Condition check occurs at the *top* of the loop.
- Unconditional branch (`b`) loops back to the evaluation point.

Translating For Loops and Indexed Bounds

Iterating through fixed numerical bounds:

C Code

```
for (int i = 0; i < 10; i++) {
  sum += i;
}
```

AArch64 Assembly Translation

```
mov x1, #0   // i = 0
mov x2, #0   // sum = 0
for_loop:
  cmp x1, #10
  b.ge for_end
  add x2, x2, x1
  add x1, x1, #1
  b for_loop
for_end:
```

Translating Do-While Loops

Unlike while loops, do-while loops evaluate their condition at the *end* of the block:

- Guaranteed to execute the loop body at least once.
- Assembly places the conditional branch at the bottom, pointing back to the loop start.
- Avoids an extra initial branch instruction required by top-checked loops.

Optimizing Loop Counters with CBZ

Using specialized opcodes for compact loop decrementing:

- Instead of separate `cmp` and `b.eq` instructions, a counter can be decremented using `subs` and tested in one step via `cbnz`:

```
loop_top:
  sub x0, x0, #1      // decrement counter
  cbnz x0, loop_top   // repeat if not zero
```

- Reduces instruction count and improves pipeline performance.

Section 06

Summary & Self-Test

Topic 7 Comprehensive Summary

- **Program Counter**: Controls execution sequencing; branching modifies the `PC` to jump to target labels.
- **Condition Flags**: Instructions like `cmp` update `NZCV` flags in `PSTATE` via silent subtraction.
- **Branch Instructions**: Unconditional branches (`b`) and conditional branches (`b.eq`, `b.ne`, etc.) handle directional jumps.
- **Specialized Opcodes**: `cbz` and `cbnz` provide efficient zero-comparison branching without altering flags.
- **Flow Translation**: High-level loops and selection structures translate cleanly into label-and-branch assembly patterns.

Self-Test: Questions 1 & 2

Question 1: The CMP Instruction

What underlying operation does the `cmp x0, x1` instruction perform, and how does it affect the processor state without modifying general-purpose registers?

Question 2: Conditional Branch Flags

After executing `cmp x5, #10` followed by `b.ne error_label`, which condition flag is inspected by the processor to decide whether to take the branch?

Self-Test: Solutions 1 & 2

Solution 1

`cmp` performs an internal subtraction ($\mathtt{x0} - \mathtt{x1}$), discards the calculated numeric result, and updates the `NZCV` condition flags in `PSTATE` based on the outcome.

Solution 2

It inspects the **Zero (`Z`) flag**. If $Z == 0$ (meaning the subtraction result was non-zero, hence values were not equal), the branch is taken.

Self-Test: Questions 3 & 4

Question 3: CBZ Advantage

What is the primary architectural advantage of using `cbz x0, label` over doing a separate `cmp x0, #0` followed by `b.eq label`?

Question 4: Loop Translation

When translating a high-level `while` loop into AArch64 assembly, why is an unconditional branch (`b`) typically placed at the end of the loop body?

Self-Test: Solutions 3 & 4

Solution 3

cbz combines the comparison and branch into a single instruction (reducing code footprint) and **does not modify condition flags**, preserving existing flag states.

Solution 4

The unconditional branch forces execution to jump backward to the loop condition evaluation point at the top, creating the repetition cycle required for iteration.

Self-Test: Questions 5 & 6

Question 5: Do-While vs. While Loops

Where is the condition check placed in a compiled `do-while` loop compared to a standard `while` loop, and what is the execution benefit?

Question 6: Symbolic Labels

Why does AArch64 use symbolic labels for branch targets instead of absolute memory addresses in source code?

Self-Test: Solutions 5 & 6

Solution 5

In a `do-while` loop, the check is at the bottom, ensuring the block runs at least once and saving an initial branch instruction. A `while` loop checks at the top.

Solution 6

Symbolic labels allow the assembler and linker to compute PC-relative offsets automatically, making code position-independent and immune to shifting memory layouts.

Review: Common Condition Suffixes

Suffix	Mnemonic Example	Logical Meaning / Flag Test
`eq`	`b.eq label`	Equal ($Z == 1$)
`ne`	`b.ne label`	Not Equal ($Z == 0$)
`lt`	`b.lt label`	Less Than (signed)
`ge`	`b.ge label`	Greater than or Equal (signed)
`hi`	`b.hi label`	Higher (unsigned greater than)

Wrap-Up and Next Steps

Transition to Topic 8

Building upon branching and control flow mechanics, the next module examines:

- **Procedures and the Stack**: Managing modular software execution and runtime memory spaces.
- **Procedure Calls and Returns**: Utilizing branch-with-link (`bl`) and return (`ret`) instructions.
- **Stack Organization**: Structuring stack frames, frame pointers (`x29`), and stack pointer alignments.
- **Parameter Passing**: Distributing arguments and return values across registers (`x0`–`x7`).
- **Calling Conventions**: Enforcing standards for caller-saved and callee-saved register preservation.